SHADOWMEM // MASTER DEFENSE & DEMO PLAYBOOK

Complete Architecture, Win32 Forensics, HistGradientBoosting ML, Plain-English Translation & Live Demo Guide

ROUND 1 JUDGING
HACKATHON EDITION
Live Prototype: **Port 8000**
Branch: `hacktoshadow`

PART 1: THE ELEVATOR PITCH & PLAIN-ENGLISH STORY

Judges evaluate clarity, real-world relevance, and confidence. Explain the problem using an analogy before dropping technical terms.

■■ SPOKEN SCRIPT (SAY THIS OUT LOUD — 30-SECOND ELEVATOR PITCH):

"Good morning, judges. Traditional antivirus scans files resting on your hard drive. But modern attackers don't drop files anymore—they inject malicious code directly into volatile computer RAM, operating completely invisibly. ShadowMem is a real-time memory forensics engine that audits the private virtual address space of running processes. We extract memory region permissions, inspect thread execution starts, and run a machine learning classifier to instantly flag in-memory code injections and process hollowing—before attackers can steal data or escalate privileges."

HOW TO EXPLAIN MEMORY FORENSICS TO ANYONE

The Apartment Complex Analogy (Say this if a judge looks confused by 'virtual memory'):
Think of running processes as tenants living in an apartment building. Traditional antivirus is like a security guard who only checks the luggage people carry in through the front lobby (files written to disk). If a burglar climbs through an open balcony window directly into someone's living room (fileless memory injection), the front guard never sees them. **ShadowMem is the room inspector:** we walk inside the building, check every private room, verify who is legally authorized to execute work there, and flag any unlisted strangers hiding in unbacked memory.

The 90-Second Structured Pitch Breakdown

Phase	What to Say to the Judges	Key Talking Point
1. Problem	Over 70% of enterprise breaches leverage fileless techniques, reflective DLLs, and process hollowing. Once in memory, attackers leave zero footprint on disk, bypassing file-based endpoint defenses.	**Disk AV is blind to active volatile memory.**
2. Solution	ShadowMem is a lightweight, user-mode Windows forensics scanner that interrogates process Virtual Address Descriptors (VAD) via Win32 APIs, coupled with a trained gradient boosted ML model.	**Direct memory inspection + ML scoring.**
3. Pipeline	We enumerate active PIDs, call VirtualQueryEx to map all committed regions, flag unbacked executable pages (MEM_PRIVATE + RWX), apply a JIT allowlist, and feed 7 memory features into our classifier.	**Full scan of ~130 processes in ~40 seconds.**
4. Live Demo	Today we demonstrate a live working prototype: real-time Win32 memory audit, machine learning anomaly detection, live thread suspension containment, and automated forensic dossier generation.	**Working end-to-end on Windows right now.**
5. Innovation	Existing memory dump tools (like Volatility) require gigabyte offline crash dumps. ShadowMem runs live in user-space with instant JIT heuristics, bringing deep memory forensics into an agile SOC dashboard.	**Live, agile, and automated triage.**

PART 2: COMPLETE ARCHITECTURE & DETECTION LOGIC

Judges often ask: *'Where does your data come from? What are the exact features? How do you prevent false positives from Chrome or Brave?'* Here is the exact ground truth.

The 5-Layer System Architecture

Layer	Component & Tech	What It Actually Does (File & Function)
1. Hardware & OS	Windows Win32 Kernel APIs (`kernel32.dll`)	Calls `OpenProcess`, `VirtualQueryEx`, and `ReadProcessMemory`. Queries Virtual Address Descriptors for region state (`MEM_COMMIT`), type (`MEM_PRIVATE`), and protection flags (`PAGE_EXECUTE_READWRITE`).
2. Forensics Engine	Python Backend Engine (`services/forensics.py`)	Enumerates all running processes via `psutil`. Iterates user-space memory (`0x10000` to `0x7FFFFFFEFFFF`), extracts memory statistics, and filters legitimate runtimes via the JIT allowlist.
3. Machine Learning	Scikit-Learn HistGradientBoosting (`shadowmem_model.pkl`)	Trained classifier evaluating 7 memory region features. Produces a threat probability (0.0 to 100.0). Maps anomalies into MITRE ATT&CK; categories (T1055.001 Reflective DLL, T1055.012 Hollowing).
4. API & WebSocket	FastAPI Async Gateway (`app.py`, `routers/forensics.py`)	Provides REST endpoints (`/api/v1/scan`, `/api/v1/audit`, `/api/v1/isolate/{pid}`) and bi-directional WebSocket (`/ws/live`) to stream forensic alerts to connected clients instantly.
5. Analyst Console	Next.js 14 / React Dashboard (`frontend/src/`)	Real-time SOC interface featuring Live Radar Map, Process Table, Threat Posture Score, Process Dossier Modal with memory breakdown, and JSON/PDF evidence export.

The 7 Features Used by the Machine Learning Model

Our classifier is a **Histogram-based Gradient Boosting Classifier** (`HistGradientBoostingClassifier`). It inspects these exact 7 signals for every running process:

Feature Name	What It Measures & Why It Matters	Normal Value	Malicious Indicator
`rwx_count`	Number of memory regions marked PAGE_EXECUTE_READWRITE. Legitimate software follows W^X (Write XOR Execute); memory that can be written to and executed simultaneously is a major red flag.	0 (or 1 in JIT)	**> 2 regions** (Shellcode stager)
`unbacked_exec_count`	Executable regions (MEM_PRIVATE) not backed by any DLL/EXE file on disk. Injected shellcode or unmapped DLLs reside in unbacked memory.	0 (normal apps) 1–3 (browsers)	**> 4 regions** (Reflective DLL injection)
`total_rwx_bytes`	Total byte size of all RWX memory allocations combined. Attackers allocate large staging buffers to assemble payloads.	0 bytes	**> 1 MB** (Payload staging)
`allocated_kb`	Total unbacked executable memory in kilobytes. Separates small stagers from large in-memory payloads.	0 KB	**> 512 KB** private executable space
`num_threads`	Total active threads in the process. Helps detect thread-based process injection and sudden spikes.	5–30 threads	Unusual spike after injection
`thread_unbacked_start`	Threads whose instruction pointer entry address points inside unbacked private memory instead of a known module.	0 (points into DLL)	≥ **1** (Thread executing raw shellcode)
`vad_protection_anomaly`	Detects dynamic memory protection flips (e.g., memory allocated as PAGE_READWRITE and later flipped to PAGE_EXECUTE_READ via `VirtualProtect`).	0 transitions	**> 1 flip** (Classic payload loader pattern)

THE JIT FALSE-POSITIVE SOLUTION

Crucial Judge Question: 'Why doesn't Chrome or Brave trigger a false alarm?'
Modern web browsers (Brave, Chrome, Edge), JavaScript engines (V8, Node.js), and .NET CLR runtimes use **Just-In-Time (JIT) compilation**. They legitimately allocate private memory pages, compile JavaScript into native machine code on the fly, and mark them executable.
ShadowMem's Defense: In `services/forensics.py`, we maintain a **JIT Runtime Allowlist** matching verified process signatures (`brave.exe`, `chrome.exe`, `msedge.exe`, `node.exe`, `Code.exe`). When an unbacked region is found in an allowlisted engine, ShadowMem classifies it as **'Legitimate JIT Engine'** (Threat Score = 0–25%), preventing alert fatigue while still auditing the regions for anomalous protection flips.

PART 3: COMPETITIVE COMPARISON & HONEST LIMITATIONS

Judges respect honesty far more than exaggeration. Know exactly what ShadowMem does better than other tools, and openly acknowledge where it stops.

Security Tool	Primary Detection Vector	Blind Spot / Limitation	Where ShadowMem Excels
Traditional AV (Defender, Norton)	Static file hashes, disk signatures, on-write file scanning.	Completely blind to fileless malware and in-memory shellcode stagers.	Audits volatile process address space where files never touch disk.
Enterprise EDR (CrowdStrike, SentinelOne)	Kernel-level telemetry, behavioral API hooks, event logs.	Heavyweight, expensive, closed-source, requires kernel driver.	Lightweight, runs in user-mode without kernel driver stability risks.
Memory Forensics (Volatility, Rekall)	Post-incident offline analysis of full physical RAM dumps.	Post-mortem only. Cannot perform live triage; dumps take minutes/hours.	Live, continuous triage in under 60 seconds with instant UI alerts.

10 Likely Judge Questions & Honest Answers

Judge Question	Your Confident Spoken Answer (Keep It Under 20 Seconds)
1. Can malware hide in the Windows Kernel?	*'Yes. Today ShadowMem operates strictly in user-mode via VirtualQueryEx. It does not inspect Ring 0 kernel space or rootkits. Kernel inspection via an eBPF or Windows driver is on our Phase 2 roadmap.'*
2. What happens if malware unhooks VirtualQueryEx?	*'Because ShadowMem calls VirtualQueryEx from its own dedicated process targeting remote processes, malware running inside the target process cannot alter ShadowMem's local API tables.'*
3. How do you distinguish legitimate JIT from malicious injection?	*'We combine a verified process allowlist with behavioral baseline checks. Even in JIT engines, we watch for thread entry points pointing into unbacked memory or sudden VAD protection flips from RW to RX.'*
4. Why does a scan take 40–50 seconds?	*'Our scanner synchronously audits ~130 running processes, iterating through over 20,000 memory regions per scan. Parallelizing the process queue across worker threads will reduce this to under 8 seconds.'*
5. What does the Isolate button actually do?	*'It suspends all execution threads belonging to that PID using OpenThread and SuspendThread. This immediately halts the malware from executing while preserving memory intact for forensics.'*
6. Where did you get the training dataset for your ML model?	*'The HistGradientBoosting model was trained on synthetic and historical feature distributions representing both benign application profiles and classic injection patterns (reflective DLL, shellcode stagers).'*
7. Why does the Scanned Processes count change between scans?	*'Windows processes constantly spawn and terminate in the background (browser sub-processes, background updaters). The +2 delta reflects net processes created since the previous scan.'*
8. Does ShadowMem require Administrator privileges?	*'It runs as a standard user to audit all user-mode processes. When elevated to Administrator with SeDebugPrivilege, it gains visibility into elevated and system-level processes.'*
9. Is the Docker Sandbox completed?	*'The Docker Sandbox and supply-chain scanner are currently architecture scaffolding on our roadmap. Our live, working core today is the Win32 forensics engine and ML scoring.'*
10. What is your commercial value proposition?	*'ShadowMem acts as an automated triage layer for SOC analysts, slashing incident response time from hours of manual memory dumping down to an instant interactive memory dossier.'*

PART 4: STEP-BY-STEP LIVE PROTOTYPE PRESENTATION SCRIPT

Follow these exact 7 steps during your 3-minute live judging demo. Do not jump around. Speak confidently, point at the screen, and guide the judges' eyes.

Step & Time	What to Do On Screen (Action)	What to Say Out Loud (Spoken Script)
STEP 1 **(0:00–0:20)** **The Opening Hook**	Open browser to **http://127.0.0.1:8000/**. Point to the Top Bar: • **LIVE API CONNECTED** • **WS LINK ACTIVE** • **Mode: user-mode (VirtualQueryEx)**	*'Judges, welcome to ShadowMem. What you are looking at is a live forensic dashboard running on this machine right now. Notice our connection chips at the top: our FastAPI backend and bi-directional WebSocket stream are connected live to the Windows OS.'*
STEP 2 **(0:20–0:50)** **Triggering the Scan**	Click the orange button: **'Run Forensics Scan'**. The button pulses, and scan indicator activates.	*'I am now triggering a live memory forensics audit. Right now, our Python engine is enumerating all running Windows processes. For each PID, it calls VirtualQueryEx across the virtual address space, searching for unbacked executable memory and protection transitions.'*
STEP 3 **(0:50–1:20)** **Reading the Results**	When the scan finishes (~40s), point to the **4 KPI Cards**: 1. **Scanned Processes** (e.g. 123 processes) 2. **Active Threats** (e.g. 1 detected) 3. **Injections Flagged** 4. **Last Scan Duration** (~45,000 ms)	*'The scan has completed in 45 seconds. It audited 123 active processes on my machine. 96 were completely clean, 26 were classified as legitimate JIT runtimes, and exactly 1 active threat was flagged with anomalous memory characteristics.'*
STEP 4 **(1:20–1:50)** **The Live Radar Map**	Point to the **Radar / Memory Map**. Show the dot colors: • Green = Clean • Blue/Purple = JIT Engine • Red = Malicious Injected. Click the **THREATS** filter tab.	*'Our radar visualizes the process posture in real time. Rather than drowning analysts in text logs, dots are clustered by risk. Clicking the THREATS tab isolates the suspicious process immediately.'*
STEP 5 **(1:50–2:20)** **The Process Dossier Modal**	Click the flagged threat row (e.g., **BlueStacksServices.exe**). The **Process Dossier Modal** opens. Highlight: • MITRE T1055.001 Classification • Memory Breakdown Chart • Attribution Bars (60% unbacked, 40% VAD anomaly)	*'This is our forensic dossier. Our HistGradientBoosting model scored this process at 73% threat probability. Notice the explainable attribution: it flagged 5 unbacked executable regions and 2 memory protection flips from read-write to execute. We map this directly to MITRE ATT&CK; T1055: Reflective DLL Injection.'*
STEP 6 **(2:20–2:40)** **Containment & Response**	Point to the action buttons inside the Dossier Modal: • **'Isolate Process'** (Thread Suspend) • **'Resume'** button. Explain containment.	*'If an attack is active, the analyst clicks Isolate Process. Our backend calls SuspendThread across all 15 process threads, freezing malicious execution in place while preserving RAM artifacts for digital forensics.'*
STEP 7 **(2:40–3:00)** **Live Alerting & Q&A; Close**	Navigate to **Alerts** page. Point to the alerts feed. Click **Export JSON** in the top bar. Turn to the judges for questions.	*'Finally, all telemetry is published over our WebSocket stream and exportable as JSON or PDF reports for legal evidence. ShadowMem brings live, automated, explainable memory forensics to any SOC. We are ready for your questions!'*

PRE-DEMO OPERATIONAL CHECKLIST

Pre-Demo Checklist (Do this 5 minutes before the judges arrive!):
1. **Confirm Backend Server is Running:** Open `http://127.0.0.1:8000/` in Chrome/Brave. Ensure green chips show *LIVE API CONNECTED* and *WS LINK ACTIVE*.
2. **Run One Pre-Scan:** Click *Run Forensics Scan* once beforehand so the dashboard has fresh live data ready on screen before the judges even walk up.
3. **Have Process Dossier Ready:** Know which process is flagged (e.g. `BlueStacksServices.exe` or an alert from the Alerts page) so your click is instantaneous.
4. **Keep This PDF Open on Your Phone or Second Screen:** Use this as your cheat sheet during judging.